

Data Mining: Project Report

Cycling, Cycling, and More Cycling

Group ID: 9

Federico Cioni 580718: *f.cioni12@studenti.unipi.it*

Giovanni Guerrazzi 580384: *g.guerrazzi3@studenti.unipi.it*

Davide Bulotta 596782: *d.bulotta@studenti.unipi.it*

January 2025

1 Introduction

In this data mining report we are going to show what we did and how we used techniques and notions learned during this year's course. Specifically through these, we are going to extract some of the “hidden knowledge” present in the two datasets given. The first representing cyclists who have competed in races, and the second representing the races themselves. We will start by understanding how these datasets are formed, go to study them, understand the problems and possible patterns we can find there and then move on to create new features in such a way as to do clustering and more. Then, after various rework, we will go on to utilize Machine Learning models in order to do predictions.

2 Data Understanding

2.1 Datasets Structure

Before we start working on the two datasets, we need to study and understand their structure so that we have a complete view of the problem we are going to deal with. The two datasets are both *data matrices*:

- **Cyclists dataset**
- **Races dataset**

2.2 Cyclist Dataset

The cyclist dataset is characterized by *6134 cyclists*. Each cyclist has a total of *6 different features*. After careful analysis, we can thus identify the type of attributes we find in the cyclist dataset:

- *_url* is a **Nominal attribute**: It represents the cyclist's url where information pertaining to the cyclist can be found.
- *name* is a **Nominal attribute**: It represents the name of the cyclist.
- *birth_year* is a **Numerical attribute**: It represents the date of birth of the cyclist.
- *weight* is a **Numerical attribute**: It represents the weight of the cyclist.
- *height* is a **Numerical attribute**: It represents the height of the cyclist.
- *nationality* is a **Nominal attribute**: It represents the nationality of the cyclist.

2.3 Races Dataset

The races dataset is significantly larger, comprising *589865 races*, each characterized by *18 features*.

- *_url* is a **Nominal attribute**: It represents the race's url where information pertaining to the race can be found.
- *name* is a **Nominal attribute**: It represents the race name.
- *points* is a **Ordinal attribute**: It represent the prestige of a race. The greater the value of points the greater the prestige of the races.
- *uci_points* is a **Ordinal attribute**: It represents an alternative to points.
- *length* is a **Numerical attribute**: It represents the length in meters of the race.
- *climb_total* is a **Numerical attribute**: It represents the total meters climbed during the race.
- *profile* is a **Ordinal attribute**: It represents the profile of the race in order of increasing difficult from 1.0 (easier) to 5.0 (harder).
- *startlist_quality* is a **Numerical attribute**: It represents how strong are the participants at the race. The higher the value the stronger the cyclists are.
- *average_temperature* is a **Numerical attribute**: It represents the average temperature of the race.
- *date* is a **Numerical attribute**: It represents the race date.
- *position* is a **Ordinal attribute**: It represents the position the cyclist arrived in.
- *cyclist* is a **Nominal attribute**: It represents the name of the cyclist.
- *cyclist_age* is a **Numerical attribute**: It represents the age of the cyclist.
- *is_tarmac* is a **Symmetric Binary attribute**: It represents if the type of surface of the race is tarmac or not.
- *is_cobbled* is a **Symmetric Binary attribute**: It represents if the type of surface of the race is cobbled or not.
- *is_gravel* is a **Symmetric Binary attribute**: It represents if the type of surface of the race is gravel or not.
- *cyclist_team* is a **Nominal attribute**: It represents the name of the cyclist's team.
- *delta* is a **Numerical attribute**: It represents how many seconds after the first placed did the cyclist get to the finish.

2.4 Feature Selection

Once we realized the structure and type of data we had in front of us, we decided to study the features to understand which ones were truly useful and impactful for future researches.

2.4.1 Null Values

First we analyzed how many null values the two datasets contained (Table 1 and 2).

An observation made in the cyclists' dataset, as seen in table 1, is that about 50% of the data regarding "*weight*" and "*height*" values are missing, since we considered weight and height to be important characteristics, this was something to study more.

In the race dataset about 25% of "*climb_total*", "*profile*" and "*cyclist_team*" are missing. 57% of "*uci_points*" corresponding to 338779 values are missing and also the column of "*average_temperature*" is practically empty (559932 missing values out of 589865), at this point we already decided to drop this feature. The null values for "*points*" and "*cyclist_age*" are negligible as they are minimal (447 the former and 113 the latter).

<i>Feature</i>	<i>Null values</i>
_url	0
name	0
birth_year	13
weight	3056
height	2991
nationality	1

Total size: 6134

Table 1: Cyclists dataset null values

<i>Feature</i>	<i>Null values</i>
_url	0
name	0
points	477
uci_points	338779
length	0
climb_total	147045
profile	148194
startlist_quality	0
average_temperature	559932
date	0
position	0
cyclist	0
cyclist_age	113
is_tarmac	0
is_cobbled	0
is_gravel	0
cyclist_team	159161
delta	0

Total size: 589865

Table 2: Races dataset null values

2.4.2 Data Consistency

Another step needed before the actual feature selection was to spot inconsistencies in the dataset, we wanted to be sure that we had no inconsistent values before selecting or dropping features. As we checked the cyclist dataset the main concern was to find duplicate “_url” or “name”, this could have caused trouble in the future. After an initial look at the cyclist dataset, we could see that there were as many as 7 repeating names, and this prompted us to check whether indeed these were duplicate names or whether indeed they were homonyms and thus different cyclists.

We ended up finding homonymy cases (Table 11 in appendix), but only for the names, because the “_url” value was different in each case, so we decided to use it as identifier of the cyclist.

We proceeded to verify the consistency of the data in the race dataset and this was a bit more articulated.

Since the race dataset contained an entry for each cyclist in each circuit (identified by the “_url”), we identified features that had to be identical for each cyclist participating in that race; in particular we made this analysis for the following features:

- points
- uci_points
- length
- climb_total
- profile
- startlist_quality
- date
- is_tarmac
- is_cobbled
- is_gravel

We are not considering the average temperature because that feature was useless to analyze. We ended up finding no inconsistencies for each case.

Another consistency check was on the numerical values, to see if there were any anomaly values present within the two datasets. This analysis of ours made us notice that the value “delta” present within the race dataset had some few outliers: as many as 86 rows presented a negative value, a value that is definitely wrong because delta should represent how many seconds after the first placed did the cyclist get to the finish and therefore should always be a positive value. However, this is not a problem since these rows do not even represent 1% of the races dataset, but we were

doubting about the usefulness of the delta: first it resulted having only 2836 distinct values, for a feature like that we expected much more variety, also it had the same value for many cyclists consecutively, and for 469,003 rows (almost 80% of the dataset), it was 0. This was definitely an anomaly that made the delta useless.

Other consistency checks like this were made on every feature, in particular, we were looking for negative values, as we did not expect there to be any (e.g. negative length, negative climb total.), but we couldn't find any anomalies like that.

Another check was on the *position*, in particular we checked that for each race we have no duplicates or values outside the range of the number of participants.

Instead, during a consistency check of the values '*is_cobbled*' and '*is_gravel*', we found that they were always 'false', which made them essentially useless.

2.4.3 Feature Selection and Data Imputation

Since we verified the consistency of the data, we were in the phase to decide which features to drop and which to use for our studies.

In the first instance, as already mentioned earlier, we deemed it appropriate to eliminate the following features even before going deeper into the analysis:

- ***average_temperature***: dropped because almost all values are NaN
- ***is_cobbled***: false for each entry
- ***is_gravel***: false for each entry
- ***delta***: anomalous value

We started considering the cyclist dataset; the only issue here was that the ***weight*** and ***height*** variables were missing for about 50% of the data. However, we believed they could be useful for future studies, so we worked to determine how to fill the missing data. The first idea was to check the high correlation between weight and height and, if possible, impute one value based on the other. As expected, the correlation turned out to be fairly high: 0.72 for Pearson correlation and 0.71 for Spearman correlation. Unfortunately there were 2984 rows where both values were null (out of 3056 null rows values for weight and 2991 for height), making this solution unfeasible.

The strategy we adopted was to fill those values using the distribution of the values based on the assumption that weight and height are picked from a Gaussian distribution, here we can see the distribution before the imputation:

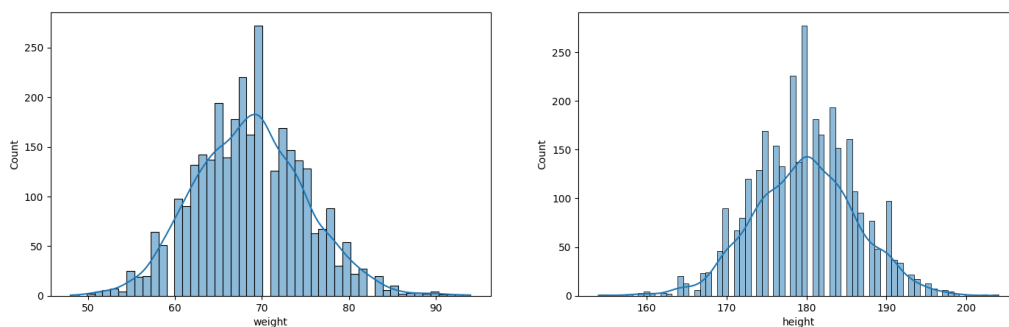


Figure 1: Distribution of weight and height before the imputation

here we can see the distribution after the imputation:

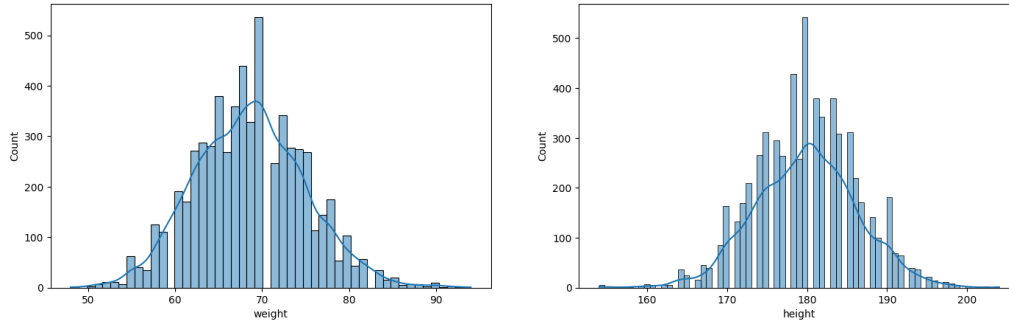


Figure 2: Distribution of weight and height before the imputation

As we can see, the distribution is maintained.

Regarding the race dataset, our main concerns were about the following features:

- *uci_points*
- *climb_total*
- *profile*
- *cyclist_team*

The reason is simple: these features were the only ones with a significant amount of NaN values. However, they could be crucial for future studies, so they required a more in-depth analysis. We started studying *uci_points* and since its definition suggested a high correlation with *points*, we decided to check it. The Pearson correlation was of 0.86, but the Kendall and Spearman correlations were respectively 0.49 and 0.54, so we decided to plot a table and check some of the strange points we obtained (fig. 3), then we also made an histplot (fig. 4)

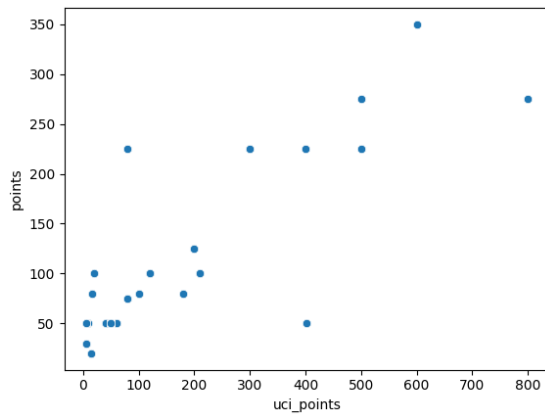


Figure 3: Plot of points and uci_points

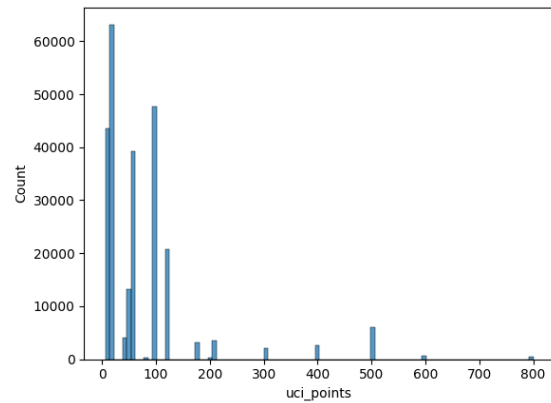


Figure 4: Histplot of uci_points

We noticed that a lot of values were very small, in fact 43.390 rows had a *uci_points* value of 6, when points was 50 each time. This and the fact that almost 57% of the data were NaN convinced us to drop this feature and to take only *points* as a way to represent the prestige of the race.

We proceeded to analyze *cyclist_team*. Cyclist team is a feature that had a reasonable amount of null values, a possibility was treating missing values as a cyclist with no team, but after a brief search online it seems like a cyclist cannot race without a team in most important races such as Tour de France, so in this first phase we decided to drop the column.

The next features we analyzed were *climb_total* and *profile*, those two features had basically the same amount of NaN values (circa 25%), but in this case there were some rows where only one value was NaN and the other was not, in fact the percentage where both values were NaN was 19.4%.

The reason we were checking where both values were NaN is that, since the profile provides an indication of the race profile, which is correlated with length and climb_total, it made sense to try imputing the missing values using the non-missing features in the same row.

Since profile is an Ordinal attribute we deemed it appropriate to calculate the correlation between it and the previously mentioned features.

The correlation between *profile* and *climb_total* was reasonably high, and there was also some correlation with *length*. Therefore, we decided to try imputing *profile* using these two features. To do this we trained a classifier, we tried a KNNClassifier, a Random Forest and a Multi Layer Perceptron. We decided to also check the performances of the classifiers, to do so we took the training set which consisted in the rows where all three values were present and split it into train and validation set, to avoid leakage (since rows had repeating values for each race) we ordered the races by date and split the data avoiding shuffle. Unfortunately, the best results (obtained by the Random Forest Classifier) were disappointing and made us leave this path (Table 12 in appendix). Since we wanted to maintain at least the *profile* feature, our only choice was to drop the *climb_total* feature and to also drop rows where *profile* was NaN. In this way, we ended up losing 25.1% of the data. The final step to obtain a dataset with no missing values was to drop the rows that presented some NaN values (e.g. *cyclist_age*, *points*..). Here is the final list of deleted features:

- *uci_points*
- *climb_total*
- *average_temperature*
- *cyclist_team*
- *is_cobbled*
- *is_gravel*

2.5 Anomalies and Outliers Detection

When we finally obtained a dataset with no missing values we started working on anomalies and outliers detection. This step was needed in order to set up the dataset for future studies like clustering.

To check anomalous values what we used were box plots and hist plots, so that we could study each feature individually. After we checked all the plots we decided to study the most interesting ones and in order to avoid showing too many graphs we decided to only talk about them. In general we didn't find too many outliers, but we observed some behaviors in the plots that were useful to obtain knowledge.

A thing worth mentioning is that since we observed that a competition like Tour de France was composed by several circuits, from now on we decided to treat each circuit as a single race.

The first observation made was on the *length* plot:

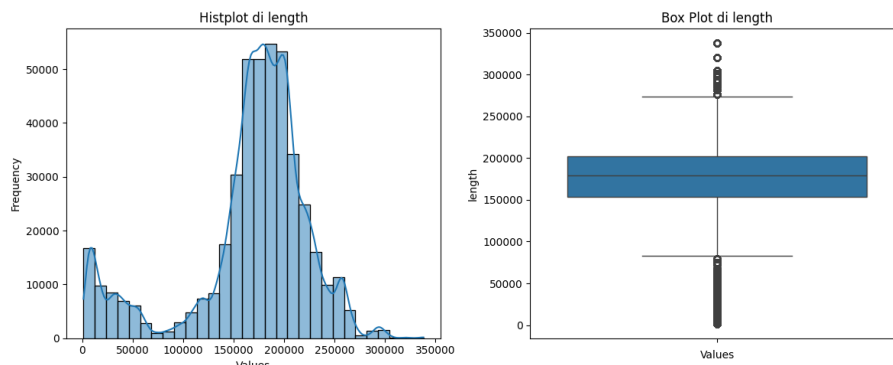


Figure 5: Length box plot and hist plot

As we can see from figure 6, the feature presented two distinct distributions, that was the main reason that produced some data that was considered outliers. After printing some of the low length races, we noticed a pattern in the name: some of the lowest length races had 'prologue' or 'stage-1' in the name. After searching online for specific low length races, we found out that they are part of a 'individual time trial' or 'prologue'. Individual time trials are often used as stages in stage races such as the Grand Tours; these vary from short prologue time trials over no more than eight kilometers (designed to create an attacking racing style earlier in the race) to longer distance events over flat or rolling courses, to timed ascents of mountain roads (mountain time trial)[2]. This explains the sort of bimodal distribution we see in the histogram. Another observation was made on the *startlist_quality* plots, where we found out a similar behavior.

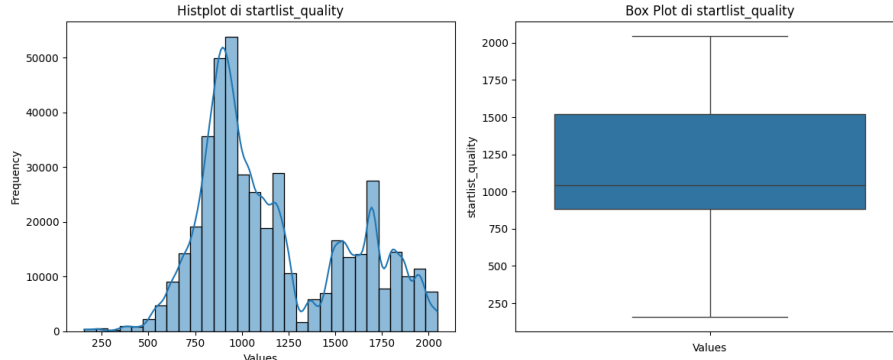


Figure 6: Length box plot and hist plot

After studying some high startlist races, we found out that the bimodal distribution was related to important races that attracted some of the best cyclists, the first 4 were: Tour de France, World Championships ME - Road Race, Milano-Sanremo, Clásica Ciclista San Sebastián.

We also studied some high points races, but we obtained different names with respect to top startlist ones, for example the first 4 are: World Championships ME - Road Race, Ronde van Vlaanderen - Tour des Flandres ME, Paris-Roubaix, Il Lombardia. Extended ranks can be found on the Project Notebook. We noticed that high points races reflected the importance of the race, for example 'Il Lombardia' also known as 'Giro di Lombardia' is notably one of the most important one-day races[1].

3 Data Transformation

In this section, we will discuss Data Transformation, which involves data preparation and a more in-depth visualization of the dataset, including the creation of new features.

3.1 Data Preparation and Visualization

The next step in our analysis was to explore more the data through a complete visualization backed by a data preparation for the clustering task. We started creating features that were useful for visualization and also plotting graphs that we considered useful for obtaining knowledge. For brevity, we present only the most significant visualizations rather than displaying all available graphs. Before starting, we merged the two datasets using the cyclist's URL as the key, as we had previously ensured that it was a valid identifier.

The first thing we did was to plot the correlation matrix of the resulting dataset, hoping to find some interesting observations (matrices can be found on the appendix).

Some observations we made:

- *year* is correlated with *birth_year* because the age of race participants is always within the same range and increases linearly.

- *startlist_quality* and *points* show a light correlation due to the fact that high points competitions are made by top tier cyclists, but this is not always the case.
- There is a slight correlation between *startlist_quality* and *year* because the dataset includes a greater number of races starting from the year 2000, potentially including lower-prestige events. In contrast, races prior to 2000 consist only of those with high-quality startlists.

Next we started working on some visualization, for example we were trying to identify the most frequent nationalities among cyclists who participated in the races, as this appeared to be an interesting aspect to examine:

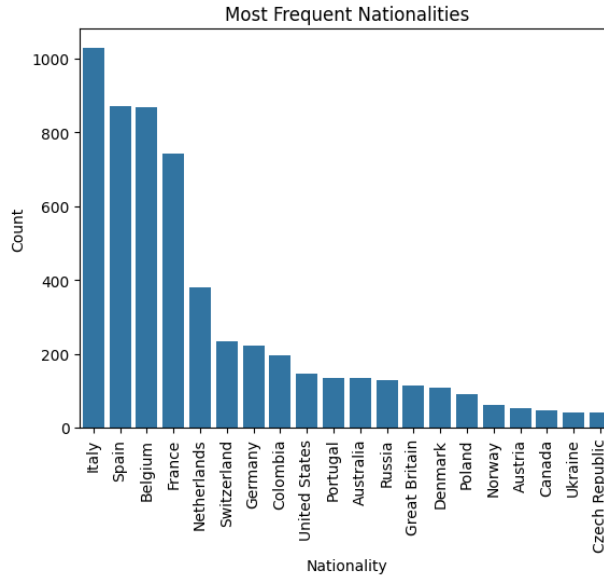


Figure 7: Most frequent nationalities

The cyclists dataset includes cyclists from *72 different nationalities*, with the **Italian nationality being the most represented** with 1029 Italian cyclists followed from Spain, Belgium and France.

Then, we wanted to see during which period of the year most competitions took place. To do this, we created a new feature, '*season*', which took the value of the season in which the race occurred. We ended up discovering the following:

- Spring: 44.28% of races
- Summer: 39.44% of races
- Autumn: 14.42% of races
- Winter: 1.84% of races

We also checked the races for each season:

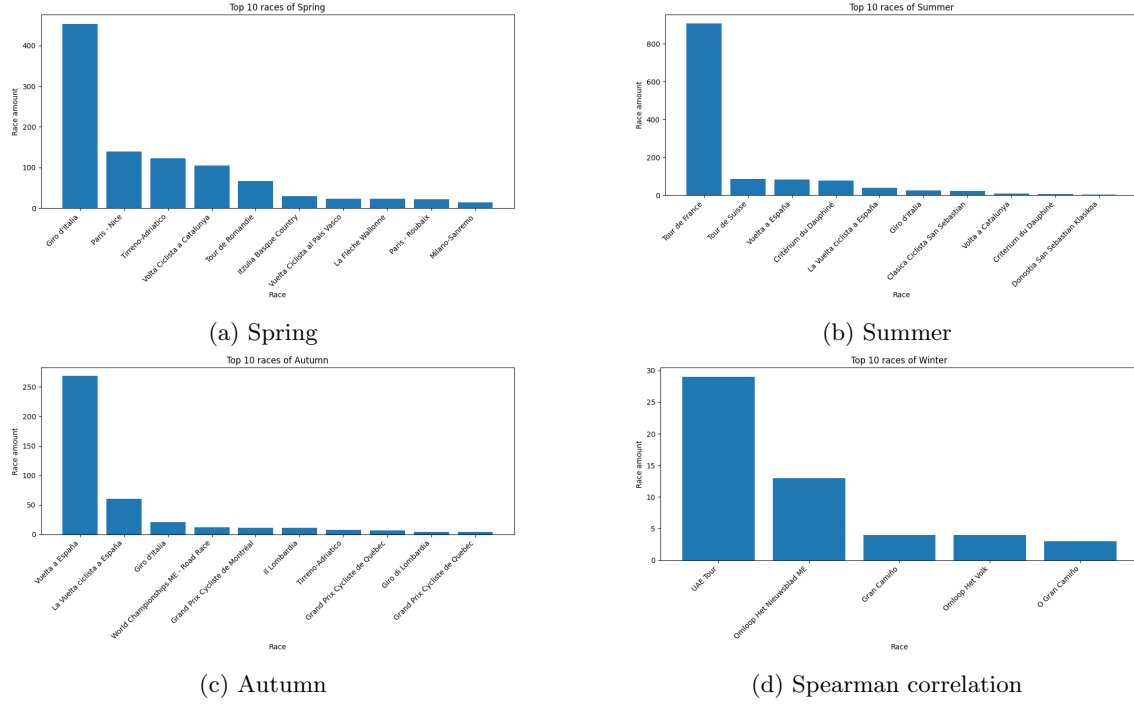


Figure 8: Seasons

From what we can see from the plots:

- Giro d'Italia is made in Spring
- Tour de France is made in Autumn
- Vuelta d'Espana is a strange one, because it appears in 3 different seasons, but it seems that it was originally held in the spring, usually late April, with a few editions held in June in the 1940s. In 1995, however, the race moved to September to avoid direct competition with the Giro d'Italia, held in May. That's why it appears in multiple seasons[3].

3.2 New Features

In this phase, we explored several potential new features that could have enhanced the analysis and modeling of cyclist performance. These features aimed to provide deeper insights into various aspects of a cyclist's abilities, considering not only their performance in individual races but also factors such as **consistency**, **profile-specific performance**, and overall **career longevity**. By integrating these new features, we hoped to create a more comprehensive understanding of what determines success in cycling, enabling more accurate predictions and assessments of cyclist capabilities across different conditions and over time. Additionally, in our mind, creating features based on race profiles or seasons was particularly valuable for clustering, as it allows for grouping cyclists with similar performance characteristics across specific race types or race periods. This can help identify patterns and distinctions that may not be immediately apparent when considering only overall race results. Below, we describe each of these features and the rationale behind their inclusion in the dataset.

- **adjusted_points:** It is a measure of how good a cyclist is based on his position, the number of races he participated in, and the points of the race. The average position is not enough because if a cyclist's average position is 2 but he participated in only 1 race, we can't be sure that he is a good cyclist. The same is true for the average position of multiple races: if a cyclist's average position is 2 and he participated in 30 races, but each race has a low points score, it is different from a top-tier cyclist who participated in 30 races with high scores and

has an average position of 2. We also wanted to give a measure of points earned by a cyclist. The formula combines everything in this way:

$$adjusted_points = \frac{points}{\sqrt{position + 1}}$$

The square root is useful to "decay" the number of points assigned to a cyclist in a non-linear way in order to give a good amount of credit also to the second cyclist, the third etc..

- **BMI (Body Mass Index)**: Calculate BMI for each cyclist using weight and height. This could be correlated with performance metrics, especially on different terrain types or difficulty levels.
- **scaled_position**: Scale the finish position to a [0, 1] range:

$$scaled_position = \frac{position - \min(position)}{\max(position) - \min(position)}$$

- **scaled_adjusted_points**: It is the same as *adjusted_points*, but we use *scaled_position*:

$$scaled_adjusted_points = \frac{points}{\sqrt{scaled_position + 1}}$$

After the creation of the new features we proceeded to create a cyclist dataset which consisted in a total of 4477 cyclists, aggregating some of the features we had. In particular we summed all the values for the *adjusted_points* features for each cyclist in order to have something like a resume of a career of a cyclist. We also computed some averages and standard deviations:

- **average_position**: Average finish position across all races in which the cyclist participated: This could indicate the cyclist's general ranking capability, but it favors cyclists who made very few races or easy races and placed in top positions.
- **avg_position_profile.x**: This feature represents the average finish position of a cyclist specifically in races with a given profile. It helps assess how well a cyclist performs in different types of race terrains or conditions, and provides insight into their specialization in certain race profiles.
- **season.x.adp**: It is a measure of how good a cyclist is in a given season. The formula is the same as '*adjusted_points*', but it is calculated only with respect to the season of the race. We end up with 4 different features, one for each season where all the adjusted points for the cyclist in each season are summed.
- **points_std**: Points standard deviation, the standard deviation of the points of the races in which the cyclist participated. A low standard deviation could mean a career made exclusively in the same kind of races.
- **position_std**: Position standard deviation, the standard deviation of the position of the cyclist. A low standard deviation could mean a consistent career.
- **age_std**: Age standard deviation, the standard deviation of the cyclist's age. A high standard deviation in this case could mean a longer career.
- **average_cyclist_age**: The average age of the cyclist across all the races he made.
- **average_normalized_position_points**: Normalize position by dividing it by points, which gives a relative position score that accounts for race prestige or difficulty. Then make the average.
- **average_normalized_position_startlist**: Normalize position by dividing it by startlist, which gives a relative position score that accounts for race prestige or difficulty. Then make the average.

- ***total_races***: The amount of total races a cyclist participated to (sum).
- ***average_profile***: The average race profile of a cyclist, which could be useful for determining the types of races the cyclist has participated in the most.

We also checked for the correlations, but all the high value correlations were predictable and didn't add any useful informations.

Some considerations were made about these new features while we were searching for outliers and anomalies:

- We have a lot of cyclists who made very few races, and very few cyclists who made a lot. Only 31 made +600 circuits and 446 made only 1 circuit. Putting a lower threshold resulted in a loss of a big part of cyclists, so we avoided it.
- Outliers for the *adjusted_points* feature, are primarily associated with highly performing cyclists. These "extraordinary" cyclists are often characterized by exceptional achievement. Notable examples (the top 5 cyclists for *adjusted_points*):
 - Alejandro Valverde
 - Peter Sagan
 - Philippe Gilbert
 - Greg Van Avermaet
 - Vincenzo Nibali

After a brief search online we noticed that these cyclists were very famous and acknowledged for their performances

- the standard deviations showed unbalance toward 0 since we had a lot of cyclists who made 1 or few more races.

4 Clustering

Here we present an analysis of cyclist performance using clustering techniques. The primary methods explored were K-Means, X-Means, Hierarchical Clustering, and DBSCAN.

The analysis focused on selecting features that could capture the essential aspects of cyclist performance, such as average positions across race profiles, consistency in results, and adjusted metrics accounting for race difficulty and participation. By applying these techniques, we aimed to understand the inherent structure of the data and identify distinct subgroups of cyclists.

4.1 Clustering with K-Means

4.1.1 Clustering with Race Profiles

One of the initial clustering configurations considered average positions across various race profiles, including flat, hilly, mountainous, and high-altitude finishes. While metrics such as the Sum of Squared Errors (SSE) and silhouette scores initially suggested an optimal $K = 4$, closer examination revealed that $K = 3$ provided a more interpretable structure, identifying three distinct clusters.

The results highlighted significant differences in cyclist abilities, a plot can be seen in image 9. A first cluster (Green) grouped cyclists with consistent performances across multiple profiles, while the second cluster (Blue) represented individuals with slightly less consistent but still solid performances. The third cluster included cyclists with average performance across most race types. This analysis underscored the importance of race profiles in distinguishing performance levels.

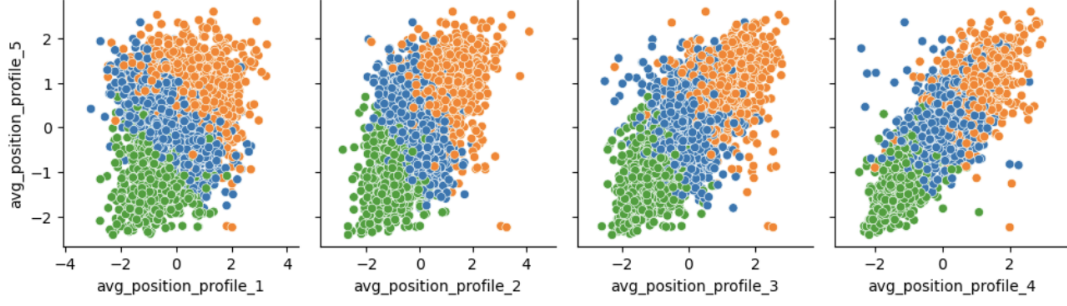


Figure 9: Clustering on race profiles

4.1.2 Incorporating Adjusted Points

Building on the initial analysis, we introduced `adjusted_points` as a feature. This metric combines a cyclist's position, race participation, and the competitiveness of events, providing a more comprehensive measure of performance. Using $K = 3$, this approach further refined the clustering results.

The top-performing cyclists formed a distinct cluster (orange one), characterized by high adjusted points and consistent results. Notably, this group included a standout individual, identified as Alejandro Valverde, a renowned cyclist with exceptional achievements. The remaining clusters captured cyclists with moderate and average performances, emphasizing the role of race participation and competitiveness in distinguishing elite performers.

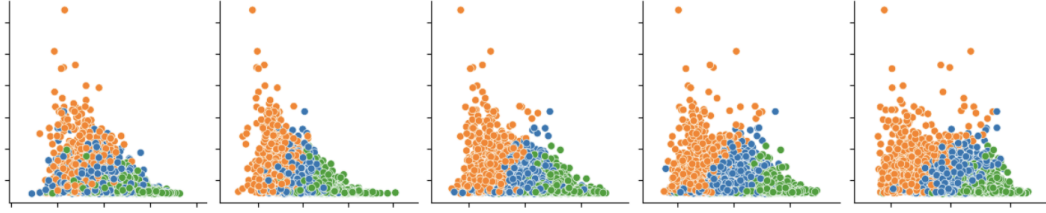


Figure 10: Clustering on adjusted points (axis y = adjusted-points and axes x = avg_position_profile_x)

4.1.3 Evaluating Variability and Consistency

To explore the consistency of performance, we incorporated the standard deviations of positions (`position_std`) and points (`points_std`) into the clustering process. This analysis used $K = 4$ to better account for the variability in results.

The clusters revealed different levels of consistency among cyclists. High-performing individuals with low variability formed one group, while another group included cyclists with moderate adjusted points but higher variability, indicating occasional strong performances interspersed with weaker results. A 3D scatter plot (image 11) provided a clearer visualization of these relationships, allowing for a more understanding of the clusters.

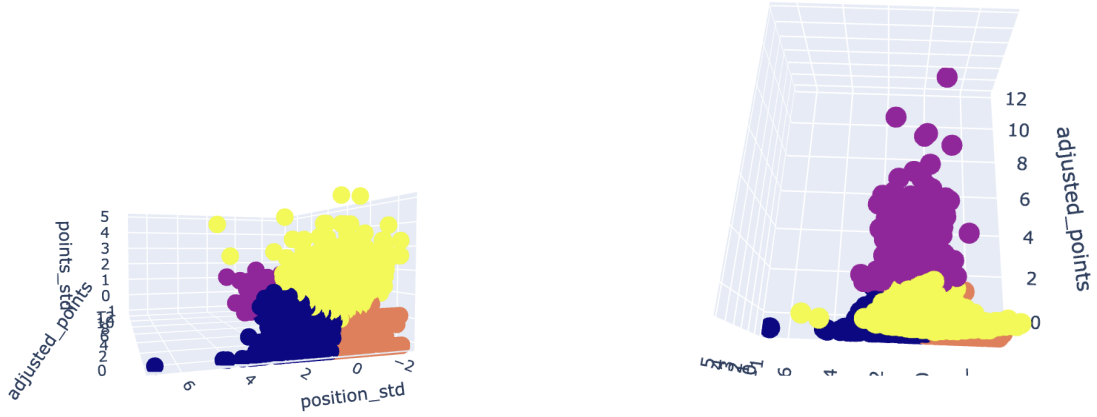


Figure 11: 3D view of the clustering

4.1.4 Filtering by Adjusted Points

To focus on high-performing cyclists, we filtered the dataset to include only those with *adjusted_points* greater than a certain threshold. However, after multiple attempts, this approach did not yield any additional useful insights.

4.1.5 Seasonal Performance Analysis

To capture seasonal variations in performance, we introduced features such as `season_1_adp`, `season_2_adp`, and so on, alongside consistency metrics. This analysis provided insights into how cyclists perform across different periods of the year.

The results highlighted two key patterns. The first cluster included cyclists with exceptional but rare performances, suggesting a smaller group of high-impact individuals during specific seasons. In contrast, the second cluster represented cyclists with greater variability across seasons, indicating inconsistency despite moderate overall adjusted points. This analysis added depth to our understanding of performance dynamics over time.

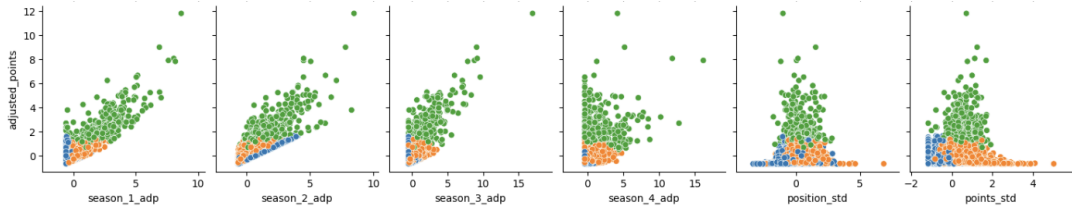


Figure 12: Clustering on seasons

4.2 Clustering with X-Means

X-Means clustering provides a more flexible and dynamic approach compared to K-Means, as it automatically determines the number of clusters based on the dataset. This flexibility allows for better capturing of the variability and complexity inherent in cyclist performance data. Below, we outline the results of various clustering configurations using X-Means, highlighting key patterns and insights.

4.2.1 Performance Across Race Profiles

Using features such as `avg_position_profile_1` through `avg_position_profile_5`, the X-Means algorithm identified six clusters. This segmentation revealed distinct groups that reflect differences

in cyclists' abilities and consistency across various race types.

The results show that one cluster, likely representing elite cyclists, consisted of individuals with the best average positions across all profiles. These cyclists demonstrated consistent high performance regardless of race type. In contrast, several mid-range clusters captured cyclists with moderate consistency but without standout results in specific profiles. Finally, a separate cluster included cyclists with poorer average positions, particularly in challenging profiles such as high-mountain and uphill finishes, highlighting the lowest-performing group.

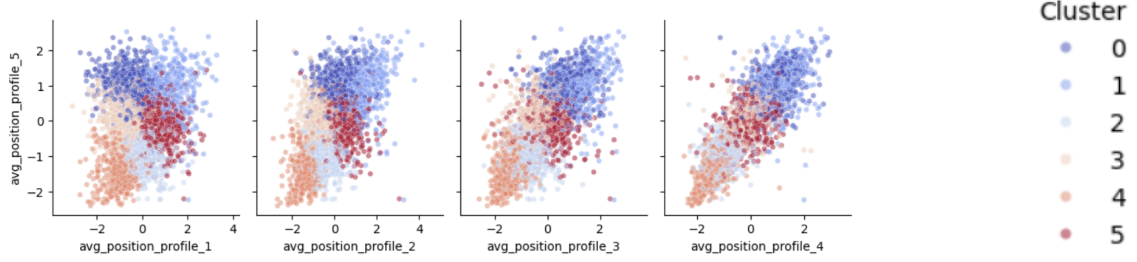


Figure 13: Clustering on different profiles

4.2.2 Detailed Segmentation Using Adjusted Points and Race Profiles

In a more detailed analysis, X-Means identified ten clusters using `adjusted_points` and average positions across the five race profiles. This high-resolution segmentation revealed a spectrum of performance levels and race specializations.

Elite cyclists formed distinct clusters, excelling in challenging terrains like mountainous and high-mountain profiles. These individuals displayed high adjusted points and consistently strong average positions. Other clusters included cyclists with solid but less exceptional results, often performing better in less demanding profiles such as flat or hilly races. Additionally, some clusters revealed greater variability in performance, with cyclists showing inconsistencies across race types. At the lower end, clusters represented cyclists with weaker results and lower adjusted points, highlighting the under performances in the dataset, in particular the orange cluster is the cluster of cyclists that performed bad in almost each race.

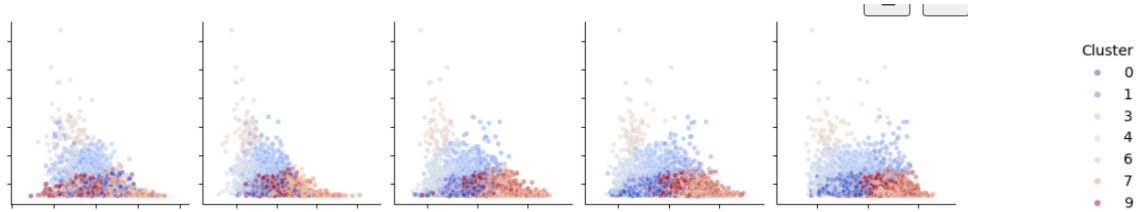


Figure 14: Clustering based on adjusted points and race profiles (axis y = adjusted-points and axes x = avg_position_profile_x)

4.2.3 Clustering Performance with Variability Metrics

Using `adjusted_points`, `position_std`, and `points_std`, X-Means again identified ten clusters. This configuration emphasized differences in consistency and variability among cyclists.

Cyclists with high adjusted points and low variability were grouped into clusters representing strong and reliable performers. Other clusters exhibited higher variability, capturing cyclists with fluctuating results across races. At the lower end, clusters included individuals with low adjusted points and minimal variability, reflecting consistently weaker performances or lower participation.

4.2.4 Refining Clusters by Filtering Adjusted Points

By filtering out cyclists with `adjusted_points` below 271, the X-Means algorithm provided a clearer stratification of performance. With ten clusters, this refined analysis distinguished between elite, mid-level, and under performing groups while highlighting specialization and variability across race types.

Interestingly, cyclists with mid-range adjusted points exhibited significant variability within their clusters. These individuals performed better in certain profiles but struggled in others, reflecting a lack of consistency. Meanwhile, clusters with low adjusted points showed consistently poor results across all race types. This filtering step improved the clarity of the analysis, focusing on high-performing and relevant cyclists.

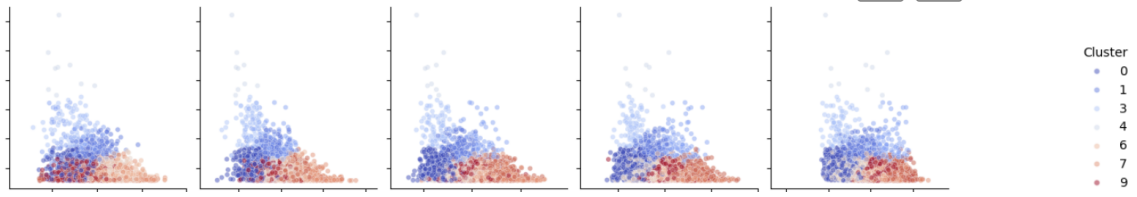


Figure 15: Clustering on adjusted points with values over 271 (axis y = adjusted-points and axes x = avg_position_profile_x)

4.2.5 Seasonal Performance Analysis

Using seasonal adjusted points (`season_1_adp` through `season_4_adp`), alongside `position_std`, `points_std`, and `adjusted_points`, X-Means identified six clusters. Compared to the results from K-Means, this approach offered finer differentiation among low and medium-level cyclists.

The top-performing group, characterized by consistently high adjusted points across all seasons, remained largely unchanged. However, X-Means provided a more nuanced segmentation of lower and mid-level performers, separating those with moderate results from consistently weaker cyclists. This improved granularity allowed for a clearer understanding of how cyclists' performances vary across seasons, highlighting differences in reliability and specialization.

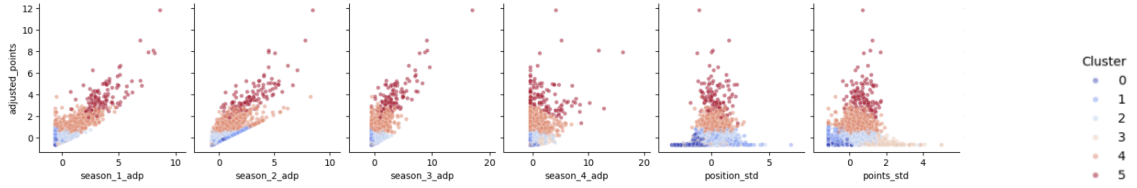


Figure 16: Clustering based on performance for season

4.3 Analysis of Hierarchical Clustering

Hierarchical clustering provides a unique approach to grouping data by progressively building a tree-like structure (dendrogram). Unlike K-Means or X-Means, it does not require a predefined number of clusters, offering flexibility and interpretability. This section presents the results of hierarchical clustering applied to cyclist performance data, showcasing how it captures both overall groupings and subtle nuances in performance and consistency.

Figure 17 shows the dendrogram created with Ward's linkage, illustrating how the data naturally splits into clusters at different distances. By cutting at the dashed red line, we get several groups that capture different performance patterns. In Figure 18, these same clusters are displayed in scatter plots, each color representing a distinct group. Some clusters (e.g., green) show lower

adjusted_points with tighter distributions, while others (like blue or orange) spread over broader ranges across various race profiles.

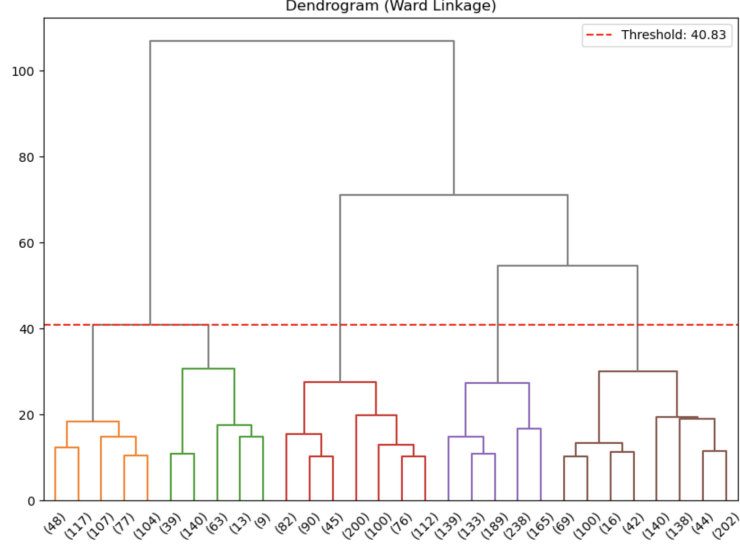


Figure 17: Dendrogram obtained with Hierarchical clustering

A key benefit of hierarchical clustering is its tree-like structure, which reveals how groups are related or how they merge as we move up the dendrogram. This can be especially useful for specialized analysis or when exploring deeper connections among subgroups. In our case, though, the hierarchical method mostly supports our earlier insights, confirming a few top-performing cyclists, a main mid-level group, and some underperforming or inconsistent clusters.

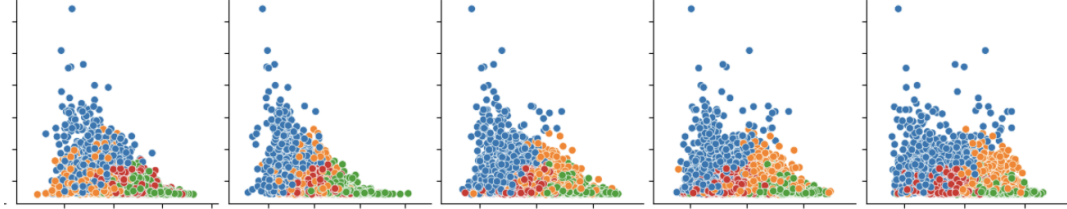


Figure 18: Scatter plot of the Hierarchical cluster (axis y = adjusted-points and axes x = avg_position_profile_x)

4.4 Conclusions

We analyzed cyclist performance using K-Means, X-Means, and Hierarchical clustering. Each method highlighted distinct patterns, with high-performing cyclists consistently forming stable clusters across all approaches. Mid-level and lower-performing cyclists showed more variability, with X-Means and Hierarchical clustering providing finer differentiation.

DBSCAN, however, was excluded from the final analyses due to its unsuitability for this dataset. Unlike K-Means or Hierarchical clustering, DBSCAN relies on density-based assumptions, which did not align with the structure of features like `avg_position_profiles`. Preliminary tests, shown in the included figure 19, demonstrated its inability to separate meaningful clusters, reaffirming the need to match algorithms to the data's characteristics.

In summary, the chosen clustering methods effectively captured performance trends and variability. Metrics like adjusted points, seasonal data, and variability measures proved crucial for meaningful segmentation. While DBSCAN was less applicable here, the overall analysis highlights the importance of selecting suitable clustering methods to uncover insights in complex datasets.

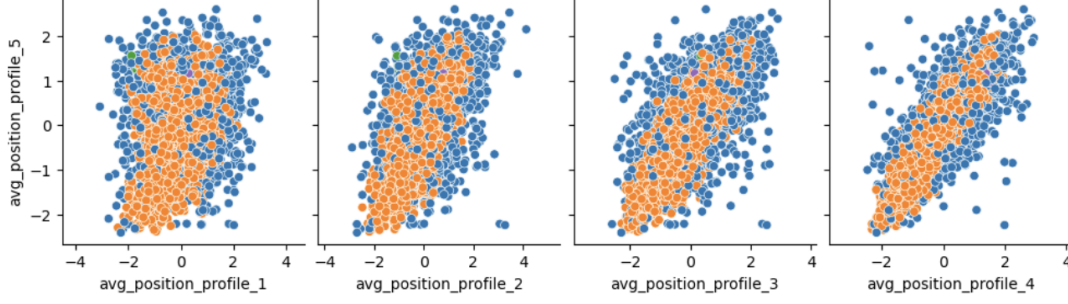


Figure 19: Clustering on DBSCAN

5 Prediction and Explanation

For the prediction part, we needed to determine whether a cyclist would place in the top 20 for a specific race.

We already knew that, with the raw dataset, this task was unfeasible because the features did not provide enough information to identify whether a cyclist would finish in the top 20. We thought a good approach would be to use data from previous races to build a history of the cyclist's past performances, as this is typically how we assess whether a cyclist has a strong chance of performing well. The logic used to construct such a dataset involved ordering the raw data by date and creating features based on past performances, excluding the race currently being analyzed.

5.1 Prediction's New Features

We created two datasets, the difference is on how *position* is treated, we have one version with the raw position and one that normalizes it with respect to the number of participants or the *startlist_quality*.

- ***avg_position***: The average position of a cyclist in the previous races.
- ***avg_position_x***: The average position of a cyclist in the previous x races. When x races are not available, consider the maximum possible amount but ensure there is a minimum given by $x - 1$. The new cyclists with start with NaN values due to the fact that we have no history about them.
- ***races_amount***: The number of races completed by a cyclist.
- ***performance_wma***: A moving average of the cyclist's position in the previous races, considering a window of 5 races.
- ***top20_consistency***: The number of times a cyclist ended up in the top 20 in the previous races.
- ***races_dby_consistency***: Top 20 consistency divided by *races_amount*. To avoid division by 0, *top20_series* is set to 1 when it is 0.
- ***prev_position_x***: The position of the cyclist x races ago.

Some adjustments were made in a normalized version of the dataset:

- ***position***: Now position is normalized with respect to the number of participants (position divided by number of participants).
- ***position_by_startlist***: A new position metric that takes into account the *startlist_quality* to normalize the position (higher startlist, lower position score) computed by dividing the position by the startlist score.
- ***avg_position_startlist***: The average of the previous races but using *position_by_startlist*.
- ***avg_position_x_startlist***: The average of the previous x races but using *position_by_startlist*.

5.2 Working Pipeline

Once the two datasets were created, the next step was to go and divide the datasets into *training* and *test* sets. Specifically, we used all races held by cyclists up to and including 2022 as the training dataset for our models and all races after 2022 as the test dataset. The models we selected for this purpose are listed below:

- Gaussian Naive Bayes
- Decision Tree
- Random Forest
- Multi-Layer Perceptron
- Support Vector Machine

For the first three models, we created a pipeline designed to process both the normalized and non normalized datasets, leveraging the most relevant and valuable features for training. Features like height, weight, date of birth, and others were deemed irrelevant for predicting whether a cyclist would finish in the top 20 in a given race. Therefore, these features were excluded from the training process. Additionally, we experimented with varying the features included in the training and testing datasets to assess whether this could lead to improved results.

We then made various attempts at pipeline refinement such as model training with Grid-Search and Cross Validation and techniques such as oversampling and undersampling.

These are the features we selected for the **non-normalized dataset**:

Feature 1	Feature 2	Feature 3	Feature 4	Feature 5	Feature 6	Feature 7	Feature 8	Feature 9
avg_position	avg_position_1	avg_position_2	avg_position_3	avg_position_4	performance_wma	top20_consistency	racess_dby_consistency	participants

Table 3: Features dataset not normalized

These are the features we selected for the **normalized dataset**:

Feature 1	Feature 2	Feature 3	Feature 4	Feature 5
avg_position_startlist	avg_position_1_startlist	avg_position_2_startlist	avg_position_3_startlist	avg_position_4_startlist

Table 4: First part of the normalized dataset with the first 5 features

Feature 6	Feature 7	Feature 8	Feature 9
performance_wma	top20_consistency	racess_dby_consistency	participants

Table 5: Second part of the dataset with the remaining 4 features

The results we obtained for each model are shown in the following sections.

5.3 Gaussian Naive Bayes

With this model we did not perform a grid search since there are no real hyperparameters to select from, but since it is a probabilistic model that works better with data that assume a Gaussian distribution we decided to use a *StandardScaler* in order to work also with the standardized datasets as well. We trained the model on both the normalized and un-normalized datasets, going for each to vary the features that composed them, and the results we noticed were about that similar between them: in particular, there was no difference between the standardized and unstandardized data, and even among the un-normalized dataset there were no particular differences in F1-score except for the accuracy, which turned out to be better in the model trained on the normalized dataset. Moreover, the models trained on the datasets where oversampling and undersampling techniques had been employed were going to worsen the results by going to lower the accuracy not bringing improvements on the F1-score of the top20 anyway. In the images 20 and 21 we see the confusion matrices of the model that performed better and in the table 6 the best results on both datasets.

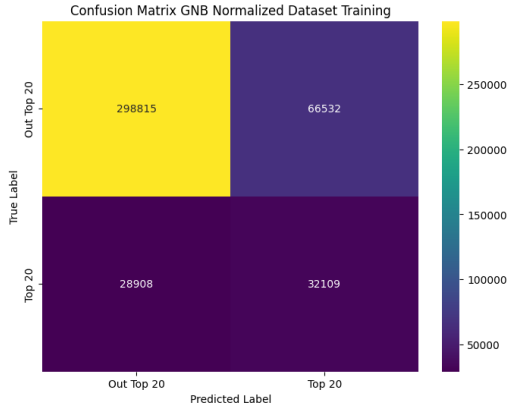


Figure 20: Confusion Matrix Training

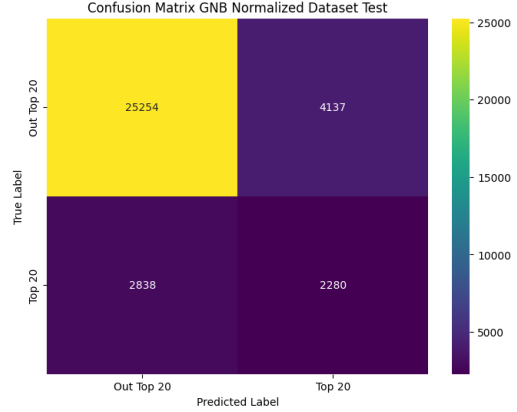


Figure 21: Confusion Matrix Test

<i>Gaussian Naïve Bayes</i>	Dataset	Accuracy	F1-Score 0	F1-Score 1
Training	Not normalized dataset	0.79	0.87	0.41
Test	Not normalized dataset	0.73	0.83	0.39
Training	Normalized dataset	0.78	0.86	0.40
Test	Normalized dataset	0.80	0.88	0.40

Table 6: Best Gaussian Naive Bayes results

5.4 Decision Tree

As for the Decision Tree, during the training phase, we performed a grid search with 5-fold cross-validation to find the model with the best performance. These are the hyperparameters on which the grid search was executed:

- **criterion:** ['gini', 'entropy']
- **splitter:** ['best', 'random']
- **max_depth:** [None, 10, 20, 30]
- **min_samples_split:** [2, 5, 10]
- **min_samples_leaf:** [1, 2, 5]
- **class_weight:** [None, 'balanced']

The best hyperparameters can be found on the appendix.

Regarding the Decision Tree trained on the un-normalized dataset what we noticed was that the results were extremely similar to each other. The F1-score remained more or less unchanged on 0.41 as well as the accuracy of the models on 0.69 even going to vary the various input parameters of the dataset. Speaking instead of the normalized one instead, the best results were obtained where we went to consider only *avg_position_startlist* and *avg_position_startlist_1* while still obtaining results roughly similar to those with the not normalized dataset.

Below are the confusion matrices obtained from the best model on the normalized dataset (figures 22 and 23) and the table 7 with the best models on both datasets:

<i>Decision Tree</i>	Dataset	Accuracy	F1-Score 0	F1-Score 1
Training	Not normalized dataset	0.72	0.82	0.42
Test	Not normalized dataset	0.69	0.79	0.41
Training	Normalized dataset	0.73	0.82	0.40
Test	Normalized dataset	0.74	0.83	0.40

Table 7: Best Decision Tree results

Again, oversampling and undersampling techniques did not lead to an increase in F1-score and thus to an overall improvement in the models but only led to a decrease in accuracy.

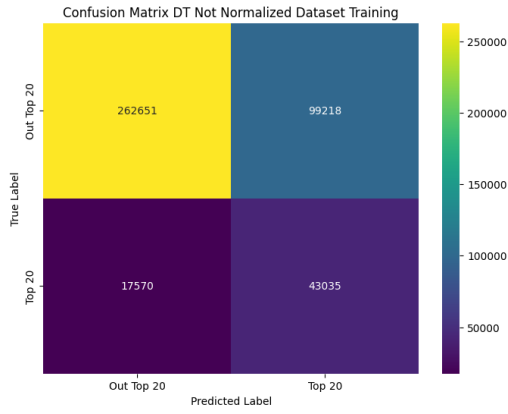


Figure 22: Confusion Matrix Training

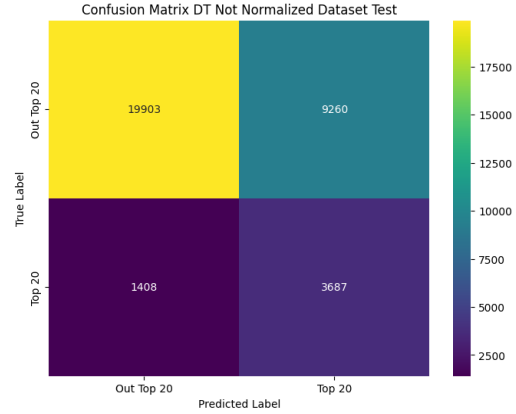


Figure 23: Confusion Matrix Test

5.5 Random Forest

Using the Random Forest instead, the parameters on which we went to perform a Grid Search were as follows:

- **n_estimators:** [50, 100, 200]
- **criterion:** ['gini', 'entropy']
- **max_depth:** [None, 10, 20, 30]
- **min_samples_split:** [2, 5, 10]
- **min_samples_leaf:** [1, 2, 5]
- **class_weight:** [None, 'balanced']

Regarding the RandomForest trained on the un-normalized dataset the results reported an F1-score that ranged from a 0.40 to a 0.42 with the accuracy settling on 69% even going to vary the features of the dataset. Another time the best results were obtained by going to consider fewer input features specifically by considering only *avg_position* *avg_position_1* and *avg_position_2*.

Going to train the model on the normalized dataset instead what we found was a marked improvement in accuracy going from a 69% to a 76% (the best) while still maintaining an F1-score of 0.41. Here unlike before the results did not change as the number of features considered increased. Again the oversampling and undersampling techniques did not lead to better results in fact most of the time the accuracy of the models worsened slightly. Below are the confusion matrices of the model that performed best of all and the table with the results of the best models on both datasets:

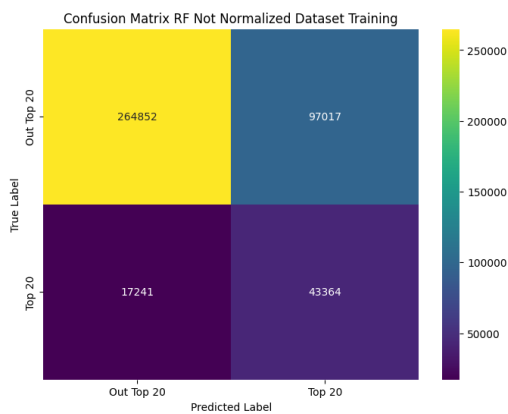


Figure 24: Confusion Matrix Training

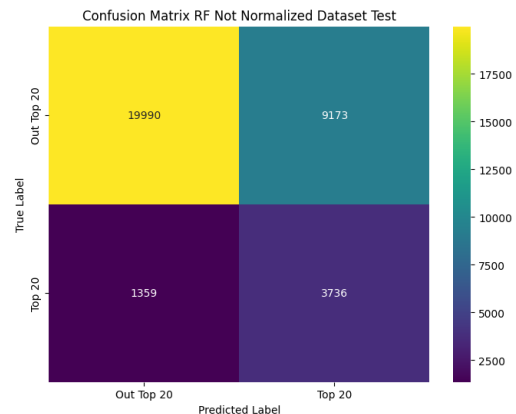


Figure 25: Confusion Matrix Test

<i>Random Forest</i>	Dataset	Accuracy	F1-Score 0	F1-Score 1
Training	Not normalized dataset	0.73	0.82	0.43
Test	Not normalized dataset	0.69	0.79	0.42
Training	Normalized dataset	0.75	0.84	0.42
Test	Normalized dataset	0.76	0.85	0.41

Table 8: Best Random Forest results

5.6 Multi-Layer Perceptron (MLP) Classifier

To classify whether a cyclist would rank in the top 20 of a race, we trained a series of Multi-Layer Perceptron (MLP) models. This approach aimed to capture complex relationships among features such as performance-weighted moving averages and consistency metrics. Below is a summary of our methodology, along with the key results from selected network configurations.

Feature Selection Considerations

In the initial stages, we tested an extensive set of features, including all five `avg_position` variables. Although combining these with additional metrics like `performance_wma`, `top20_consistency`, and `racess_dby_consistency` increased the model’s complexity, it did not yield better performance. Ultimately, the best results were obtained by focusing on the latter three features alone. This streamlined set not only simplified the network architecture but also improved its predictive accuracy, suggesting that more features do not necessarily translate into better outcomes.

Training Procedure and Network Configuration

We implemented our MLP using *PyTorch*, defining fully connected layers with various sizes, activation functions, and optimization hyperparameters. The table below outlines typical configurations (expressed as $[3, h_1, h_2, 1]$ for hidden-layer sizes h_1 and h_2), all of which used:

- **Loss Function:** Binary Cross-Entropy with Logits (`BCEWithLogitsLoss`), optionally applying class weighting to address class imbalance.
- **Optimizer:** Stochastic Gradient Descent (SGD) with momentum.
- **Activation:** Typically `tanh`, though `ReLU` was also tested in preliminary runs.
- **Learning Rate and Penalty:** Tested a range of learning rates (e.g., 0.001, 0.008, 0.01, 0.05, 0.1) and regularization values to balance convergence speed with overfitting.
- **Batch Size:** 15,000 samples per training iteration.
- **Epochs:** Tested on [20,25,30,35,40,50,60,70,80,90,100] or more, depending on convergence behavior.

For each configuration, the dataset was split into training and test sets, with any missing values dropped prior to learning.

Prediction and Evaluation

After training, we used the trained network to predict probabilities (`sigmoid(z)`) on both the training and test sets. Rather than simply thresholding these probabilities at 0.5, we replicated the competition setting by sorting cyclists’ predicted probabilities within each `race_id` and marking the top 20 as “predicted_top20.” This approach ensures that exactly 20 cyclists are selected per race. Thresholding alone could have led to more than 20 cyclists exceeding the threshold, which would not align with the constraint that only 20 riders can be in the top 20. In fact, for a single race, we could have had much more cyclists labeled as top20, impacting on the evaluation of the results.

To assess performance, we calculated the accuracy and F1 score on both the training and test sets. The confusion matrices and classification reports were stored for further analysis.

Selected Results

Table 9 summarizes a few key configurations tested. Each row lists the network structure, main hyperparameters (activation, momentum, learning rate), and the resulting accuracy and F1 score on the test set. We report only a subset of the most notable trials, focusing on the improvements gained by adjusting hidden layer sizes and hyperparameters.

Layers	Momentum	LR	Accuracy (Test)	F1 - class 0	F1 - class 1
[3, 10, 20, 1]	0.9	0.1	0.795	0.880	0.30
[3, 15, 30, 1]	0.9	0.1	0.807	0.887	0.34
[3, 30, 60, 1]	0.9	0.1	0.810	0.889	0.35
[3, 50, 100, 1]	0.9	0.1	0.812	0.890	0.36
[3, 55, 110, 1]	0.9	0.1	0.814	0.891	0.36
[3, 55, 110, 1]	0.8	0.01	0.826	0.898	0.40
[3, 55, 110, 1]	0.8	0.1	0.816	0.893	0.37
[3, 60, 120, 1]	0.9	0.1	0.809	0.888	0.35
[3, 65, 130, 1]	0.9	0.1	0.811	0.890	0.35
[3, 70, 140, 1]	0.9	0.1	0.813	0.891	0.36
[3, 75, 150, 1]	0.9	0.1	0.810	0.889	0.35
[3, 40, 80, 1]	0.9	0.1	0.813	0.890	0.36
[3, 50, 100, 1]	0.9	0.1	0.813	0.891	0.36

Table 9: Sample of MLP configurations and performance metrics. (**LR** = learning rate).

Several trends emerged:

- Increasing hidden-layer size (e.g., from [3, 10, 20, 1] to [3, 50, 100, 1]) generally ¹ improved test set performance, suggesting a benefit from higher model capacity.
- Adjusting the momentum parameter (e.g., from 0.9 to 0.8) occasionally helped by smoothing updates, while variations in the learning rate influenced both convergence speed and overall accuracy.
- Small changes to the learning rate (e.g., from 0.01 to 0.008) sometimes yielded slight gains in F1, indicating a sensitive trade-off between stability and learning speed.

Best-Performing Model

Among the configurations tested, the MLP with architecture [3, 55, 110, 1], using the **tanh** activation function and **SGD** optimizer (momentum = 0.8, weight_decay = 0.0002, learning_rate = 0.008), achieved the highest overall performance. This model attained an **accuracy of 0.8355** on the training set and **0.8260** on the test set, accompanied by F1 scores of **0.9043** (train) and **0.8981** (test).

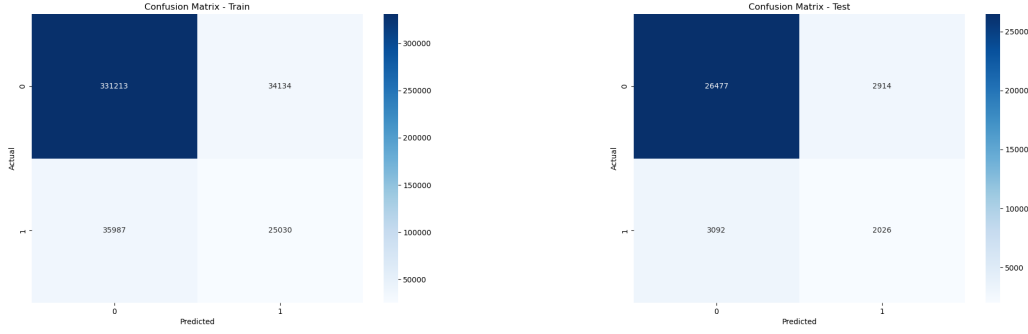
To have a clearer view on the capacity of the model it’s important to look at the F1 scores of the classes singularly: on the majority class (0), where it achieves a precision and recall near **0.90**. The minority class (1) exhibits lower absolute precision and recall values, yet the balance between these metrics still leads to an F1 score of approximately **0.40** on the test set. Notably, the weighted average across both classes remains above 0.82, reflecting the model’s efficacy in prioritizing top-20 predictions.

On the training set the majority class shows both high precision and recall, while the minority class retains a moderate F1 score of about 0.42. Overall, the accuracy and F1 score values suggest that the network generalizes well without overfitting.

Class Weights and Sampling Approaches

During experimentation, we applied class weights in the loss function to address the inherent imbalance between top-20 and non-top-20 cyclists. Contrary to expectations, however, the weighted loss

¹After 110 like size for the $[h_1]$ the performance get worst



(a) Confusion Matrix - Train

(b) Confusion Matrix - Test

Figure 26: Clustering based on performance for season

did not improve minority-class precision and, in some cases, adversely affected overall performance. We also explored under sampling and oversampling techniques to further balance the dataset, but these methods introduced additional complexities, such as distorting class distributions or reducing sample diversity. Consequently, retaining the original class proportions without applying heavy sampling strategies resulted in more consistent model performance in this particular classification task.

Normalization Tests

We also conducted experiments using a normalized version of the dataset, where features were scaled to ensure comparable ranges. However, these trials produced results that differed little from those obtained with the original, unscaled data. Given the minimal impact of normalization on key metrics such as accuracy and F1 score, we opted to proceed with the non-normalized dataset on MLP and SVM solutions.

Predictions Analysis: Alignments and Discrepancies

To illustrate how the MLP interprets the features (*performance_wma*, *top20_consistency*, *racess_dby_consistency*), we tested two cyclists, each with a distinct set of inputs, across the same race stage.

Table 10 summarizes one scenario where the model predictions aligned closely with reality, and another where the outcome diverged from expectations.

Cyclist	perf. wma	top20 cons.	racess dby_cons.	Pred. Prob.	Actual
<i>1st Test: Alignment</i>					
Walter-Planckaert	9.33	2.0	1.0	$\approx 51.96\%$	1
Bernard-Thevenet	29.66	1.0	2.0	$\approx 11.77\%$	0
<i>2nd Test: Discrepancy</i>					
Bernard-Hinault	16.0	1.0	2.0	$\approx 40.03\%$	0
Ludo-Loos	92.33	0.0	2.0	$\approx 11.77\%$	1

Table 10: Comparison of model inputs, predicted probabilities, and actual outcomes in two scenarios.

In the first test, the cyclist with moderate performance metrics receives a lower probability of placing in the top 20 and indeed fails to achieve it, while the cyclist with a higher *performance_wma*

is predicted more likely to finish in the top 20 and does so. In contrast, the second test shows a disconnect between the modeled probability and the actual outcome: despite the lower `performance_wma` suggesting a stronger chance for the first cyclist, the second unexpectedly secures the top 20 finish. These examples underline the potential nuances not recognized by our model, but which may change the the final prediction.

5.7 Support Vector Machine (SVM) Classification

In addition to the MLP-based approach, we experimented with a Support Vector Machine (SVM) classifier. While SVMs often excel in high-dimensional spaces and can handle various kernel functions, they can become computationally expensive² when dealing with large datasets. In this case, the size of our training set significantly increased training times, requiring careful selection of kernel parameters and regularization to reach a feasible solution.

Despite the computational overhead, the resulting model achieved an overall accuracy of 0.80 on the test set. However, as indicated by the classification report below, performance on the minority (positive) class remained limited in terms of precision and recall:

	precision	recall	f1-score	support
0	0.88	0.89	0.88	29391
1	0.32	0.31	0.31	5118
accuracy			0.80	34509
macro avg	0.60	0.60	0.60	34509
weighted avg	0.80	0.80	0.80	34509

5.8 Explainability: Other Considerations

Below are graphs showing us how much impact the features had in model predictions, particularly in Decision Tree and Random Forest. Graphs with all the features per dataset have been reported to get a more complete view. What we can see is that the features `avg_position_*` and `avg_position_*_startlist` in both models had less impact than features such as `performance_wma` and `racess_dby_consistency` that impact more.

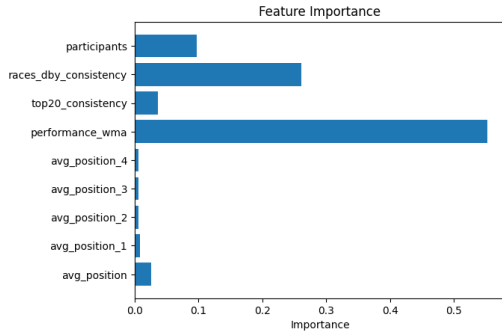


Figure 27: Feature importance Decision Tree not normalized dataset

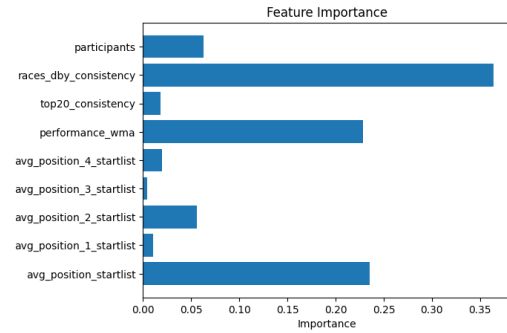


Figure 28: Feature importance Decision Tree normalized dataset

² $O(n_{\text{features}} \times n_{\text{samples}}^2)$ till $O(n_{\text{features}} \times n_{\text{samples}}^3)$

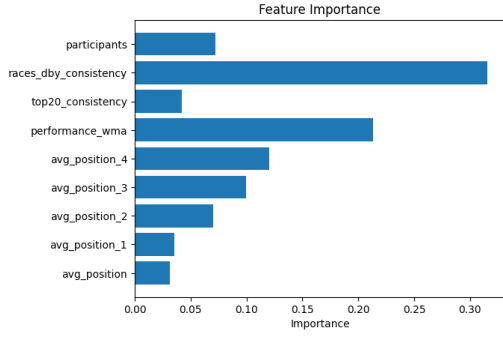


Figure 29: Feature importance Random Forest not normalized dataset

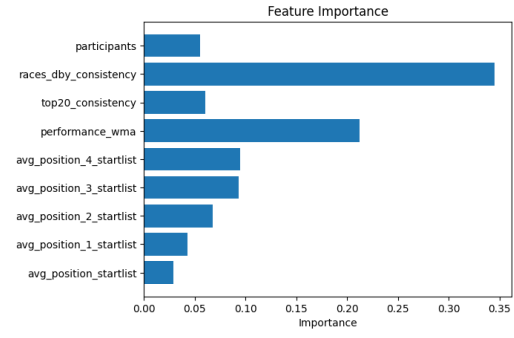


Figure 30: Feature importance Random Forest normalized dataset

In addition to the observations we previously made interacting with the models, we have also confirmed a correlation between being in the top20 and the most impactful features.

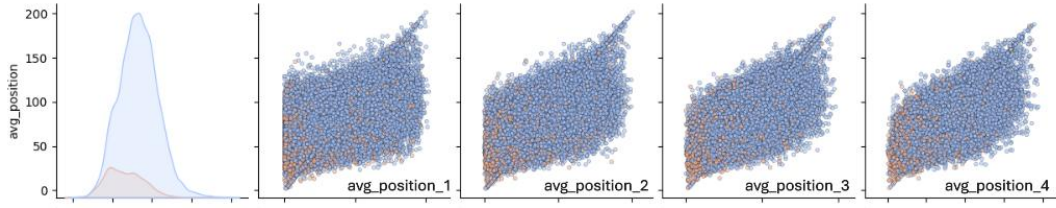


Figure 31: Pairplot of avg_position with avg_position_x (orange dots indicate a cyclist being in top20)

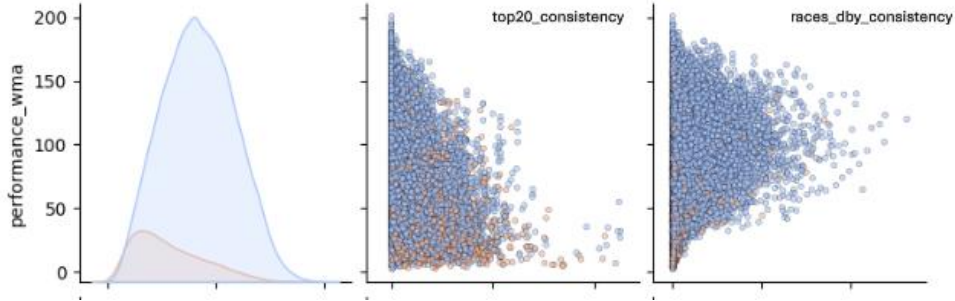


Figure 32: Pairplot (orange dots indicate a cyclist being in top20)

The scatterplots reveal a consistent trend: cyclists who frequently place in the top 20 tend to show better average positions, whether we analyze their performance over the entire dataset or restrict it to the most recent races.

races_dby_consistency and **performance_wma** are very impactful as we have seen in recent plots, and it is also confirmed by the plots in figure 32.

Unfortunately, it often happens (as the performance of the models confirms) that previous results alone are not sufficient to determine whether a cyclist will place in the top 20 or not. Clearly, there are other factors at play that we have either been unable to identify or did not have access to in our analysis.

This highlights the complexity of the problem and suggests that external variables, for example team dynamics, race-day conditions, or even psychological factors, could play a significant role in influencing a cyclist's performance. Addressing these gaps would likely require additional data or more comprehensive modeling approaches.

6 Appendix

_url	name	birth_year	weight	height	nationality
alberto-fernandez-sainz	Alberto Fernández	1981.0	NaN	NaN	Spain
alberto-fernandez-blanco	Alberto Fernández	1955.0	NaN	NaN	Spain
alessandro-pozzi2	Alessandro Pozzi	1969.0	NaN	NaN	Italy
alessandro-pozzi	Alessandro Pozzi	1954.0	NaN	NaN	Italy
andrea-peron-1	Andrea Peron	1971.0	70.0	183.0	Italy
andrea-peron	Andrea Peron	1988.0	70.0	178.0	Italy
antonio-cabello-baena	Antonio Cabello	1990.0	67.0	179.0	Spain
antonio-cabello	Antonio Cabello	1956.0	NaN	NaN	Spain
jesus-lopez23	Jesús López	1955.0	NaN	NaN	Spain
jesus-lopez-carril	Jesús López	1949.0	NaN	NaN	Spain
roman-kreuziger-sr	Roman Kreuziger	1965.0	NaN	NaN	Czech Republic
roman-kreuziger	Roman Kreuziger	1986.0	65.0	183.0	Czech Republic
sergio-dominguez-rodriguez	Sergio Domínguez	1979.0	NaN	NaN	Spain
sergio-dominguez-munoz	Sergio Domínguez	1986.0	NaN	NaN	Spain

Table 11: Duplicate name in the cyclists dataset

Class	Precision	Recall	F1-Score	Support
1.0	0.58	0.82	0.68	15810
2.0	0.55	0.44	0.49	28092
3.0	0.22	0.15	0.18	11943
4.0	0.28	0.30	0.29	8477
5.0	0.58	0.66	0.62	17496
Accuracy			0.50	81818
Macro Avg	0.44	0.47	0.45	81818
Weighted Avg	0.49	0.50	0.49	81818

Table 12: Classification Report of Random Forest of *profile* classification

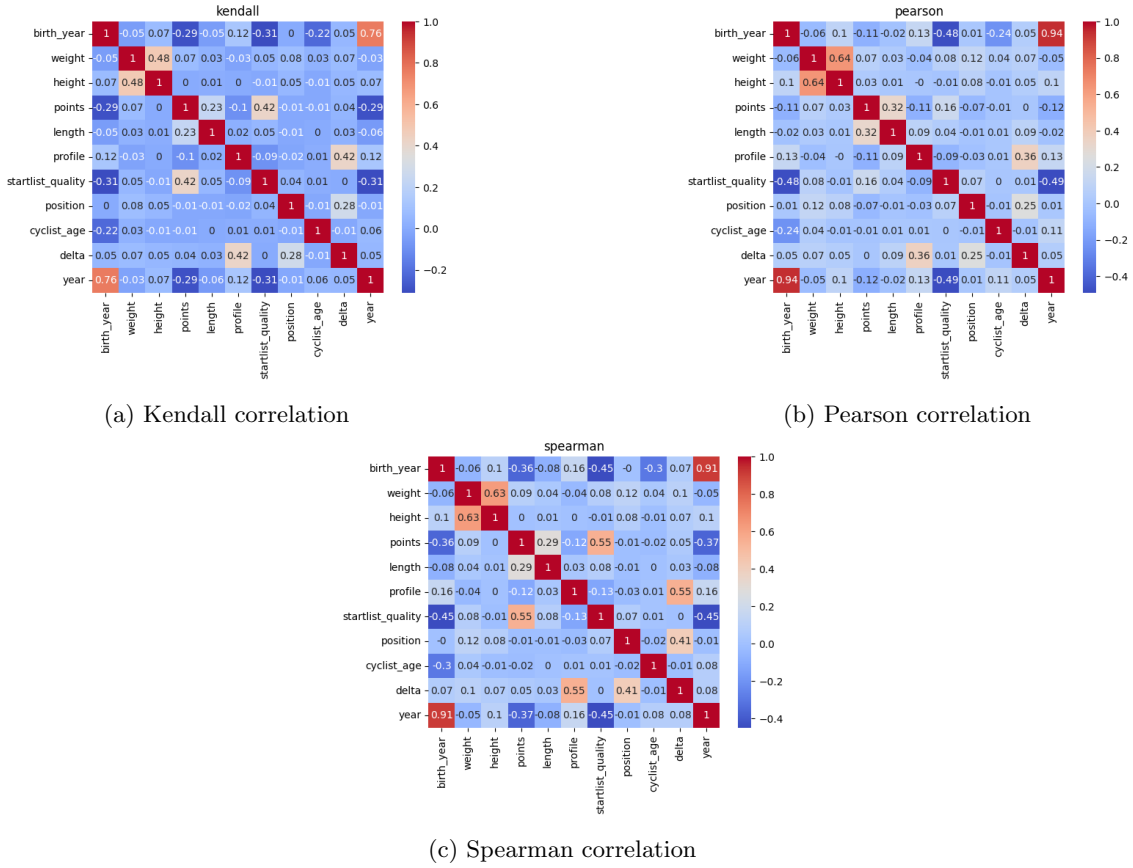


Figure 33: Comparison of correlation methods

6.1 Best hyperparameters for the Decision Tree

The best hyperparameters found for the Decision Tree trained on the *non-normalized dataset* after the grid search are the following: **'criterion': 'entropy', 'splitter': 'best', 'max_depth': 10, 'min_samples_split': 2, 'min_samples_leaf': 5, 'class_weight': 'balanced'**. Differently, the best hyperparameters found for the Decision Tree trained on the *normalized dataset* are as follows: **'criterion': 'entropy', 'splitter': 'random', 'max_depth': 10, 'min_samples_split': 2, 'min_samples_leaf': 1, 'class_weight': 'balanced'**.

6.2 Best hyperparameters for the Random Forest

The best hyperparameters found for the Random Forest trained on the *non-normalized dataset* after the grid search are the following: **'n_estimators': 200, 'criterion': 'gini', 'max_depth': 10, 'min_samples_split': 2, 'min_samples_leaf': 5, 'class_weight': 'balanced'**. Differently, the best hyperparameters found for the Random Forest trained on the *normalized dataset* are as follows: **'n_estimators': 200, 'criterion': 'gini', 'max_depth': 10, 'min_samples_split': 2, 'min_samples_leaf': 2, 'class_weight': 'balanced'**.

References

- [1] https://it.wikipedia.org/wiki/Giro_di_Lombardia
- [2] https://en.wikipedia.org/wiki/Individual_time_trial
- [3] https://it.wikipedia.org/wiki/Vuelta_a_Espa%C3%B1a